

3-Day Sprint · Team of 4 · Build-Measure-Learn Framework

1. Vision Statement

For small businesses (hotels, retail stores, etc.) who want customers to discover their inventory intuitively, **[Product Name]** is a QR-based catalog system where staff add items with rich descriptions, and customers query them in natural language ("something spicy for 2 people" / "shoes for muddy trekking") instead of scrolling menus or shelves.

2. Leap-of-Faith Assumptions (What we're actually betting on)

#	Assumption	Type	Riskiest?
1	Clients (staff) will bother writing <i>descriptive</i> attributes for items (not just name+price)	Value (supply side)	Medium
2	Customers will actually type/speak a natural-language query instead of just scrolling a list	Value (demand side)	High — this is the core bet
3	AI semantic search returns results customers perceive as <i>relevant and better than browsing</i>	Value (product)	Highest — this IS the product
4	Businesses see enough value to want to keep using it after the demo	Growth	Low (test later)

Riskiest assumption to validate in 3 days: #3 — *does semantic/descriptive search actually beat a plain list/filter for the customer?* Everything in the MVP scope should be built to test this as fast as possible, not to build a "complete product."

3. Customer Segments (two-sided)

- **Clients (supply side):** hotel staff, shop owners — add/manage items
- **Customers (demand side):** diners, shoppers — scan QR, query, decide

Two example verticals to seed test data (don't build vertical-specific logic — same schema, different content):

- 🍽️ Hotel menu: taste, mood, spice level, group size, dietary tags
 - 🦵 Footwear store: outfit style, use-case (indoor/trekking/formal), season
-

4. MVP Scope — What We ARE Building

Core loop (must work end-to-end by end of Day 2):

1. Client logs in with a simple business code → adds item (name, price, description/attributes, optional photo)
2. System auto-generates a QR code linking to that business's public catalog page
3. Customer scans QR → lands on mobile web page with a search box
4. Customer types (or use speech to text) a free-text query → AI embeds query, does semantic similarity search over items, optionally LLM re-ranks/explains → returns top 3–5 items with a one-line "why this matches"
5. Every query + result is logged for measurement

Explicitly OUT of scope for MVP (cut ruthlessly — build these only if Day 3 has spare time):

- User authentication/roles (use a simple business slug/PIN instead of real auth)
 - Payments or ordering
 - Stock/quantity tracking
 - Multi-language support
 - Native mobile app (mobile-responsive web only)
 - Business-facing analytics dashboard (just log events, don't visualize yet)
 - Editing/deleting items (add-only is enough to test the hypothesis)
-

5. Team Split (4 Members)

Role	Owner	Responsibilities
A — Backend/API	Member 1	DB schema, Business/Item CRUD, business slug system, image upload
B — AI/Search	Member 2	Embeddings pipeline, vector search, query endpoint, LLM re-rank + "why it matches" explanation
C — Frontend	Member 3	Client add-item form, customer search page, QR code generation & display, mobile-first UI
D — Product/QA/Ops	Member 4	Scope guardrails, seed data (menu + footwear), deployment, event logging, test coordination, demo/pitch prep

(If your team prefers pairing instead of hard ownership, A+B can pair on the AI-heavy backend and C+D can pair on frontend+ops — just keep one clear owner per module to avoid blocked handoffs.)

6. Suggested Tech Stack (optimized for speed, not scale)

- **Frontend:** Next.js (or Vite+React) + Tailwind — fast to build, mobile-responsive by default
 - **Backend:** FastAPI (Python) — same language as AI code, less context-switching
 - **Database:** Supabase (Postgres + built-in `pgvector`) — gives you relational data *and* vector search in one hosted service, no separate vector DB needed
 - **AI:** Claude or OpenAI embeddings for semantic search; a small LLM call for re-ranking/explaining top results, Also can use Gemini speech to text models
 - **QR generation:** `qrcode` (Python) or `qrcode.react` (JS) — trivial, don't over-engineer
 - **Hosting:** Vercel (frontend) + Render/Railway (backend), or all on one platform if the team prefers fewer moving parts
-

7. Day-by-Day Action Plan (Build → Measure → Learn)

Day 1 — BUILD: Foundations

Goal: skeleton pieces exist independently by EOD

All (1st hour): Kickoff — align on MVP scope (Section 4), lock data model, agree on API contract between A/B/C so no one blocks each other.

- **A:** Set up backend project, DB schema (`Business` , `Item`), CRUD endpoints for items
- **B:** Set up embedding pipeline; prototype semantic search on dummy hardcoded items (prove the concept works in isolation before wiring it up)
- **C:** Frontend skeleton — add-item form (client side), search page shell (customer side), QR generation working with a static URL
- **D:** Repo/CI setup, choose deployment target, write realistic seed datasets for **both** verticals (10–15 menu items, 10–15 footwear items with rich descriptions), define the 4–5 success metrics (Section 8)

EOD checkpoint: Item can be added → saved to DB → shown in a plain list (no AI yet).

Day 2 — BUILD: Core Loop + AI Integration

Goal: full end-to-end flow works, even if rough

- **A:** Finish CRUD, add image upload (optional), business slug/PIN onboarding flow
- **B:** Wire embeddings into item creation (auto-embed name+description on save); build `/query` endpoint: embed query → vector similarity search → LLM re-rank top results → generate short "why this matches" text
- **C:** Connect frontend to live APIs; QR now encodes real business URL; build search results UI showing AI explanation per result
- **D:** Deploy to staging; run both seed datasets through the live flow; start writing QA test cases; instrument event logging (query text, results shown, which result — if any — user taps)

EOD checkpoint: Someone can scan a real QR code on their phone, type "something spicy for 2 people," and get a plausible ranked answer.



Day 3 — MEASURE + LEARN + Polish

Goal: validate the riskiest assumption, not just ship code

Morning:

- Full team QA pass on both verticals with real seed data; fix obvious bugs
- **A/B:** Tune relevance (bad matches, empty results, ambiguous queries), harden the query endpoint
- **C:** Mobile polish (this is a phone-first product — test on an actual phone), loading/empty states
- **D:** Recruit 5–10 people (colleagues/friends) to actually scan the QR on their phones and try real queries; capture: did they find what they wanted? how long did it take? did they need to rephrase?

Afternoon — Build-Measure-Learn review (whole team, 30 min):

- Look at the metrics (Section 8) together
- Answer explicitly: *did semantic search beat "just scrolling a list" for our testers?*
- Decide: persevere (keep building on this direction) or pivot (e.g., maybe filters+tags beat free-text search — that's a valid learning too)

Evening:

- Final polish based on what's fixable in remaining hours
 - Prepare a short demo script + 3–5 slide pitch summarizing what was learned, not just what was built
 - Deploy final link, confirm QR codes work from a cold phone scan
-

8. Metrics to Capture (Innovation Accounting, not vanity metrics)

Metric	What it tells you
Item add completion rate & time-to-add	Is describing items too much friction for clients?
Query → relevant result rate (tester-judged)	Is the AI actually useful, or just novel?
Time-to-find (scan → satisfied)	Faster than manually browsing?
Query reformulation rate	High = search quality is weak
Quick thumbs-up/down after each query result	Cheap, real-time relevance signal

These numbers — not the demo polish — are what tell you whether to keep investing in this direction after Day 3.

9. Guardrails for the 3 Days

- **Don't build for scale.** One business = fine for MVP. Multi-tenant hardening comes later.
- **Don't build admin auth.** A shared PIN/slug is enough to prove the concept.
- **Don't polish UI before the core loop works end-to-end.** A working ugly flow beats a broken pretty one.
- **If Day 2 EOD checkpoint isn't hit, cut scope further** — e.g., drop image upload or LLM re-ranking (pure vector search is still a valid MVP) rather than slipping the timeline.

NB: WE SHOULD CONSIDER LOCAL LANGUAGE SUPPORT, ON EMBEDDING, SPEECH TO TEXT..

POSSIBLE WORK AROUNDS: TRANSLATE MAL TO ENG THEN DO AI SHITS, TRANSLATE BACK..CAN SKIP FOR MVP